

บทที่ 3

ลินุกซ์เคอร์เนลโมดูลและซิสเต็มคอลล์(System call)

3.1 ลินุกซ์เคอร์เนลโมดูล

3.1.1 ทำความรู้จักกับลินุกซ์เคอร์เนลโมดูล

ลินุกซ์เคอร์เนลโมดูลใช้ในการขยายฟังก์ชันให้กับเคอร์เนลของลินุกซ์โดยมีข้อดี คือ เป็นการโหลดข้อมูลแบบไดนามิกซึ่งทำให้ไม่ต้องทำการโหลดข้อมูลทั้งหมดโดยโหลดเพียงส่วนที่มีการใช้งานเท่านั้นและสามารถใช้งานได้เลยโดยไม่ต้องคอมไพล์เคอร์เนลใหม่

ลินุกซ์โมดูลมีฟังก์ชันพื้นฐาน 2 ฟังก์ชัน คือ

1. `int init_module(void)` /* เป็นฟังก์ชันที่ใช้ในการกำหนดค่าเริ่มต้น */
2. `void cleanup_module(void)` /* เป็นฟังก์ชันที่ใช้ในการ shutdown */

โดยทั่วไปแล้วการโหลดโมดูลต้องมีสิทธิ์เป็นรูตเท่านั้น โดยใช้คำสั่งดังนี้ :

```
#insmod module.o
```

คำสั่งนี้ส่งผลกระทบต่อระบบ คือ

- โหลดออบเจกต์ไฟล์ (object file) ขึ้นมา ในที่นี้ คือ module.o
- เรียกซิสเต็มคอลล์ (systemcall) ชื่อ `create_module` เพื่อทำการจองเนื้อที่ในหน่วยความจำ
- ซิสเต็มคอลล์ชื่อ `get_kernel_syms` จะอ้างอิงในส่วนของ Kernel-symbols
- หลังจากกำหนดค่าเริ่มต้นโดยใช้ซิสเต็มคอลล์ `init_module` แล้วก็จะทำการเอ็กซีคิวต์ (execute) ฟังก์ชัน `int init_module(void)` ต่อไป

ซึ่งรายละเอียดของ Kernel-Symbols จะขออธิบายไว้ในหัวข้อต่อไป

ตัวอย่าง ข้างล่างนี้จะเป็นตัวอย่างของการสร้างโมดูลอย่างง่าย

```
#define __KERNEL__

#define MODULE

#include <Linux/kernel.h>
#include <Linux/module.h>

int init_module(void) {
    printk("<1>Hello World\n");
    return 0;
}

void cleanup_module(void) {
    printk("<1>Bye, Bye");
}
```

สังเกตว่าในตัวอย่างข้างต้นจะใช้คำสั่ง `printk` ไม่ใช่ `printf` เนื่องจากการเขียนโปรแกรมในเคอร์เนล จะแตกต่างกับการเขียนโปรแกรมในยูสเซอร์สเปซมาก ดังนั้นเวลาการเขียนโปรแกรมในเคอร์เนลจำเป็นที่จะต้องระมัดระวังคำสั่งการใช้งานมากเนื่องจากคำสั่งเหล่านี้จะมีข้อจำกัดในการใช้งาน ดังนั้นจึงควรเรียนรู้ฟังก์ชันต่าง ๆ ในแอปพลิเคชันของ userspace ไว้เพื่อช่วยในการเจาะระบบเคอร์เนลได้

จากตัวอย่างข้างต้นสามารถคอมไพล์โดยใช้คำสั่ง

```
# gcc -c -O3 helloworld.c
```

โหลดโมดูลโดยใช้คำสั่ง

```
# insmod helloworld.o
```

เมื่อโมดูลถูกโหลดขึ้นมาและแสดงตัวอักษรขึ้นมา เราใช้คำสั่งตรวจสอบเพื่อให้เห็นว่าลิ้นุกซ์โมดูลยังอยู่ในเคอร์เนลสเปซ ดังนี้

```
# lsmod
```

Module	Pages	Used by
Helloworld	1	0

คำสั่งข้างต้นจะอ่านข้อมูลใน `/proc/modules` เพื่อโหลดโมดูลที่ชื่อ `helloworld` โดยใช้เวลาชั่วขณะหนึ่ง ส่วน `Page` เป็นข้อมูลของหน่วยความจำ (จำนวนของ `page` ที่อยู่ในโมดูล) และ `Used by` จะบอกความถี่ที่ระบบเรียกใช้โมดูล `helloworld` (อ้างอิงการนับ) เราสามารถถอดโมดูลออกจากระบบได้ หากค่าของตัวแปร `counter` มีค่าเป็น 0 โดยคำสั่งที่ใช้ย้ายคือ

```
# rmmod helloworld
```

ในลักษณะของลิ้นุกซ์โมดูลมีลักษณะคล้ายกับโปรแกรม DOS TSR รุ่นเก่า คือ สามารถจับสัญญาณอินเทอร์รัปต์ทุกสัญญาณที่เราต้องการได้ ดังเช่น WIN9X ของบริษัท Microsoft ที่มีการเรียก (call) `VxD` ซึ่งเหมือนกับ ลิ้นุกซ์โมดูลที่มีการเรียกฟังก์ชันของระบบ เรียกว่า ซิสเต็มคอลล์ (systemcalls)

3.1.2 Kernel Symbol Table

เมื่อเข้าใจพื้นฐานของซิสเต็มคอลล์และโมดูลแล้ว ยังมีจุดที่น่าสนใจและเป็นส่วนที่สำคัญมากอีกแห่งหนึ่ง คือ ตาราง Kernel-Symbol ซึ่งอยู่ในไฟล์ `/proc/ksyms` การเข้ามายังไฟล์นี้จะถือเป็นการเอ็กซ์พอร์ต (export) หรือพับลิค (public) Kernel Symbol ซึ่งสามารถแอ็กเซส (access) ลิ้นุกซ์โมดูลที่เขียนขึ้นได้

นอกจากนี้ไฟล์ `/proc/ksyms` ยังสามารถช่วยให้สามารถเห็นสิ่งที่ลิ้นุกซ์โมดูลทำการโหลดมาได้แต่ยังคงมีปัญหาอยู่ข้อหนึ่ง คือ ทุก Symbol ที่ใช้ใน ลิ้นุกซ์โมดูล (ซึ่งเหมือนกับฟังก์ชัน) จะเป็นการ export ไปยัง public ทำให้เกิดเป็นลิสต์ขึ้นในไฟล์ซึ่งอาจเป็นการบุกรุกได้ในทางหนึ่งโดยผู้บุกรุกอาจทำการซ่อนลิ้นุกซ์โมดูล นั้นไว้

ดังนั้นผู้ดูแลระบบที่มีประสบการณ์มากพอจะค้นพบ ลิ้นุกซ์โมดูล นี้และทำการลบทิ้งเพื่อป้องกันการบุกรุก

นักพัฒนาส่วนของ ลินุกซ์โมดูล สามารถใช้โค้ดข้างล่างนี้จำกัดการเอ็กซ์พอร์ตซิมโบล (export symbol) ในโมดูลได้

```
static struct symbol_table module_syms= { /*ทำการระบุตารางซิมโบล*/
#include <linux/symtab_begin.h>          /*ซิมโบลจะถูกจำกัดการเอ็กซ์พอร์ต*/
...
};

register_symtab(&module_syms); /*ทำการรีจิสเตอร์*/
```

หากไม่ต้องการให้ซิมโบลถูกเอ็กซ์พอร์ตไปยังพลับบิก (public) ควรใช้คำสั่งดังนี้ :

```
register_symtab(NULL)
ซึ่งคำสั่งนี้ต้องใส่ในส่วนของฟังก์ชัน init_module()
```

3.1.3 การเปลี่ยนจากส่วนของ Kernel ไปหน่วยความจำใน ยูสเซอร์สเปซ

จากที่หัวข้อที่ผ่านมาทำให้ทราบข้อดีในการโค้ดดิ้ง (coding) ในส่วนของเคอร์เนลสเปซแล้วแต่ก็ยังคงมีข้อเสียอยู่เหมือนกัน กล่าวคือ เมื่อซิสเต็มคอลล์รับอาร์กิวเมนต์มาจากยูสเซอร์สเปซ (ซิสเต็มคอลล์จะถูก อิมพลีเมนต์ใน wrapper เหมือนกับไลบรารี libc) แต่ลินุกซ์โมดูลนั้นรันในส่วนของเคอร์เนลสเปซ

ในหัวข้อนี้จะยกตัวอย่างของการเข้าถึงอาร์กิวเมนต์ที่ถูกจองไว้ในส่วนของยูสเซอร์สเปซจากโมดูลในเคอร์เนลสเปซได้ ดูจากตัวอย่างซิสเต็มคอลล์นี้ :

```
int sys_chdir (const char *path) /* เป็นฟังก์ชันการเปลี่ยนไดเรกทอรี */
```

เมื่อระบบทำการเรียกซิสเต็มคอลล์นี้จะหยุดการคอลล์ (call) อื่นไว้แล้วทำการตรวจสอบพาท (path) ที่ผู้ใช้ต้องการตั้งค่า ดังนั้นจึงต้องมีการใช้ค่า const char *path ซึ่งหากพยายามใช้ค่าตัวแปร path โดยตรงอาจเกิดปัญหา real problem ได้เช่นเดียวกับการ printf (“<1%s\n”, path);

ในเคอร์เนลสเปซไม่สามารถอ่านหน่วยความจำในยูสเซอร์สเปซได้ง่ายนัก แต่ก็สามารถทำได้ โดยจะมีมาโคร(macro) ที่ชื่อ get_user (pointer) ให้ทำหน้าที่ให้ฟังก์ชันพอยน์เตอร์ชี้ไปยัง *path ซึ่งช่วยให้อ่านค่า (เป็นไบต์) จากหน่วยความจำในยูสเซอร์สเปซไปยังเคอร์เนลสเปซ

ต่อไปนี้เป็นการทำงานของ plaguez ในการย้ายสตริงจากยูสเซอร์สเปซไปยังเคอร์เนลสเปซ :

```
char *strncpy_fromfs(char *dest, const char *src, int n) {
    char *tmp = src;
    int compt = 0;
    do {
        dest[compt++] = __get_user(tmp++, 1);
    }
```

```

        while ((dest[compt - 1] != '\0') && (compt != n));
        return dest;
    }
    ถ้าต้องการกลับค่าตัวแปร *path ก็สามารถใช้ส่วนของโค้ดในเคอร์เนลได้ดังนี้
    char *kernel_space_path;
    /* จองหน่วยความจำในเคอร์เนลสเปซ */
    kernel_space_path = (char *) kmalloc(100, GFP_KERNEL);
    (void) strncpy_fromfs(test, path, 20);      /* เรียกฟังก์ชันของ plagues */
    printk("<1%s\n", kernel_space_path);        /* ใช้ข้อมูลใดก็ได้ที่ต้องการ */
    kfree(test);                                /* ทำการคืนหน่วยความจำ */

```

จากโค้ดข้างต้นสามารถทำงานได้ดี สำหรับทรานซิชันทั่วไปค่อนข้างมีความซับซ้อนมากซึ่งอธิบายรายละเอียดในหัวข้อถัดไป

3.1.4 วิธีการใช้ยูสเซอร์สเปซให้เหมือนกับฟังก์ชัน

จากหัวข้อที่แล้วได้ใช้ syscall macro ในการสร้าง brk call ของตนเองขึ้นมา และฟังก์ชันไลบรารีในยูสเซอร์สเปซนั้นถูกอิมพลีเมนต์ (implement) ผ่าน syscall macro ดังซอร์สโค้ดที่แสดงใน _syscall1(...) macro ซึ่งใช้ในหัวข้อที่ 4 ในการสร้างฟังก์ชัน brk(...) (มาจาก /asm/unistd.h)

```

#define _syscall1(type,name,type1,arg1) \
    type name(type1 arg1) \
    { \
        long __res; \
        __asm__ volatile ("int $0x80" \
                           : "=a" (__res) \
                           : "0" (__NR_##name),"b" ((long)(arg1))); \
        if (__res = 0) \
            return (type) __res; \
        errno = -__res; \
        return -1; \
    }

```

ในส่วนนี้มันจะทำการเรียกอินเทอร์รัปต์ (interrupt) 0x80 กับอาร์กิวเมนต์ (argument) ที่ถูกส่งค่ามาจาก _syscall1 เพื่อใช้เรียกซิสเต็มคอลล์ที่ต้องการ(ซึ่งอาจถูกขยายเป็น _NR_name ซึ่งระบุไว้ใน /asm/unistd.h) วิธีนี้จะใช้อิมพลีเมนต์กับฟังก์ชัน brk เท่านั้น ส่วนฟังก์ชันกับอาร์กิวเมนต์อื่น ๆ ก็จะอิมพลีเมนต์ผ่าน macros อื่นเช่นกัน (_syscallX โดยที่ X เป็นจำนวนอาร์กิวเมนต์)

โดยทั่ว ๆ ไปแล้วอาจใช้วิธีอื่นในการอิมพลีเมนต์ฟังก์ชันได้ ดังจากตัวอย่างต่อไปนี้ :

```
int (*open)(char *, int, int);          /* ประกาศโปรโตไทป์ */
open = sys_call_table[SYS_open];      /* สามารถ __NR_open */
```

วิธีนี้อาจไม่ต้องใช้ syscall macro อื่นเลยเพียงแต่ต้องใช้ฟังก์ชันพอยน์เตอร์จาก sys_call_table เท่านั้นและพบว่าการสร้างยูสเซอร์ให้เหมือนฟังก์ชันโดยวิธีนี้จะมีการใช้ใน famous LKM infector โดย SVAT ซึ่งในความเป็นจริงแล้วถือว่าการแก้ปัญหาที่ดีกว่า แต่ย่อมขึ้นอยู่กับตัวผู้ใช้อย่างยิ่ง

ข้อควรระวังเมื่อทำการส่งอาร์กิวเมนต์ไปยังซิสเต็มคอลล์เหล่านี้ เพราะซิสเต็มคอลล์เหล่านี้ต้องการอาร์กิวเมนต์ใน ยูสเซอร์สเปซ ไม่ใช่จากใน เคอร์เนลสเปซ สำหรับวิธีการนำข้อมูลจาก เคอร์เนลสเปซ ไปยังหน่วยความจำ ยูสเซอร์สเปซ นั้นให้ศึกษาจากหัวข้อข้างต้นที่ได้กล่าวไว้แล้ว

ในระบบปฏิบัติการลินุกซ์นั้นจะใช้ segment selector ในการเปลี่ยนระหว่างเคอร์เนลสเปซและ ยูสเซอร์สเปซ ซึ่งอาร์กิวเมนต์ที่ใช้กับซิสเต็มคอลล์จะถูกส่งมาจากยูสเซอร์สเปซซึ่งเป็นส่วนหนึ่งใน data segment selector (DS) range

Data segment นี้สามารถทำการคืนค่าได้โดยใช้ฟังก์ชัน get_ds() จากไลบรารี asm/segment.h ดังนั้นข้อมูลที่ใช้จะเป็นพารามิเตอร์ (parameter) โดยซิสเต็มคอลล์สามารถแอ็กเซสพารามิเตอร์นี้ได้จากเคอร์เนลสเปซ หากมีการตั้งค่า segment selector ให้ใช้ได้กับ user segment โดยเคอร์เนลไปยังค่า DS ที่ต้องการ โดยสามารถใช้ฟังก์ชัน set_fs(...) ข้อควรระวัง คือ ต้องเก็บค่า FS หลังจากทำการแอ็กเซสอาร์กิวเมนต์ของซิสเต็มคอลล์แล้ว ดังตัวอย่างโค้ดบางส่วนข้างล่างนี้

```
-filename is in our kernel space; a string we just created, for example
unsigned long old_fs_value=get_fs();
set_fs(get_ds);          /*หลังจากส่วนนี้จะสามารถแอ็กเซสข้อมูลในยูสเซอร์สเปซได้*/
open(filename, O_CREAT|O_RDWR|O_EXCL, 0640);
set_fs(old_fs_value); /*restore fs...*/
```

ฟังก์ชันที่แสดงในตัวอย่างนี้ (brk, open) ถูกอิมพลีเมนต์ผ่านซิสเต็มคอลล์ตัวเดียวเท่านั้นแต่ยังมีกลุ่มของฟังก์ชันยูสเซอร์สเปซที่รวบรวมไว้ในซิสเต็มคอลล์หนึ่ง สามารถดูรายละเอียดได้ในหัวข้อที่ 2 เช่น sys_socketcall จะอิมพลีเมนต์ทุก ๆ ฟังก์ชันที่เกี่ยวกับ socket (การสร้าง, ปิด, ส่ง, รับ) ดังนั้นจึงควรระมัดระวังเมื่อสร้างฟังก์ชันขึ้นมาแล้วจะต้องคอยดูในเคอร์เนลซอร์ส (kernel source) เสมอ

3.1.5 รายชื่อของฟังก์ชันในเคอร์เนลสเปซที่ใช้บ่อย

ฟังก์ชัน printk(...) ที่ได้กล่าวถึงในขั้นต้นนั้นเป็นฟังก์ชันที่ทุกคนเคยใช้ในเคอร์เนลสเปซ ดังนั้นจึงเรียกว่า ฟังก์ชันเคอร์เนลฟังก์ชันเหล่านี้ถูกสร้างขึ้นเพื่อให้ผู้ที่ต้องการฟังก์ชันที่ซับซ้อนได้ใช้งาน ซึ่งโดยปกติแล้วผู้ใช้งานจะใช้ฟังก์ชันธรรมดาผ่านไลบรารีฟังก์ชันเท่านั้น

ตารางข้างล่างนี้แสดงรายชื่อฟังก์ชันเคอร์เนลที่สำคัญ ๆ ไว้

Function/macro	description
int sprintf (char *buf, const char *fmt,...); int vsprintf (char *buf,const char *fmt, va_list args);	ฟังก์ชันในการเก็บข้อมูลเป็น string
printf(..)	เหมือนกับฟังก์ชัน printf ในยูสเซอร์สเปซ
void *memset (void *s, char c, size_t count); void *memcpy (void *dest, const void *src, size_t count) char *bcopy (const char *src,char *dest, int count); void *memmove (void *dest, const void *src,size_t count); int memcmp (const void *cs, const void *ct, size_t count); void *memscan (void *addr, unsigned char c,size_t size);	ฟังก์ชันหน่วยความจำ
int register_syntab (struct symbol_table *intab);	ดูในตาราง 2.1
char *strcpy (char *dest, const char *src); char *strncpy (char *dest, const char *src,size_t count); char *strcat (char *dest, const char *src); char *strncat (char *dest, const char *src, size_t count); int strcmp (const char *cs, const char *ct); int strncmp (const char *cs, const char *ct,size_t count); char *strchr (const char *s, char c); size_t strlen (const char *s); size_t strlen (const char *s, size_t count); size_t strspn (const char *s, const char *accept); char *strpbrk (const char *cs, const char *ct); char *strtok (char *s,const char *ct);	ฟังก์ชันการเปรียบเทียบ string ใดๆ
unsigned long simple_strtoul (const char *cp, char **endp,unsigned int base);	กลับค่าจาก string เป็นตัวเลข

Function/macro	description
Get_user_byte (addr);	ฟังก์ชันในการเข้าถึงเซสของผู้ใช้
Put_user_byte (x,addr); Get_user_word (addr); Put_user_word (x,addr); Get_user_long (addr); Put_user_long (x,addr);	หน่วยความจำ
Suser(); Fsuser();	ตรวจสอบสิทธิ์ของ Super user
Int register_chrdev (unsigned int major,const char *name, struct file_operations *fops); Int unregister_chrdev (unsigned int major, const char *name); Int register_blkdev (unsigned int major, const char *name, struct file_operations *fops); Int unregister_blkdev (unsigned int major, const char *name);	ฟังก์ชันการรีจิสเตอร์อุปกรณ์ต่าง ๆ ..._chrdev -> character devices ..._blkdev -> block devices

ตารางที่ 3.1 แสดงรายชื่อของฟังก์ชันใน เคอร์เนลสเปซที่ใช้บ่อย

ฟังก์ชันเหล่านี้บางส่วนอาจเป็นฟังก์ชันที่สร้างขึ้นมาเพื่อใช้ประโยชน์โดยใช้วิธีการที่ได้กล่าวถึงในส่วนที่ 5 (ฟังก์ชันที่มีความสำคัญมากในตาราง คือ ฟังก์ชันการเปรียบเทียบ string)

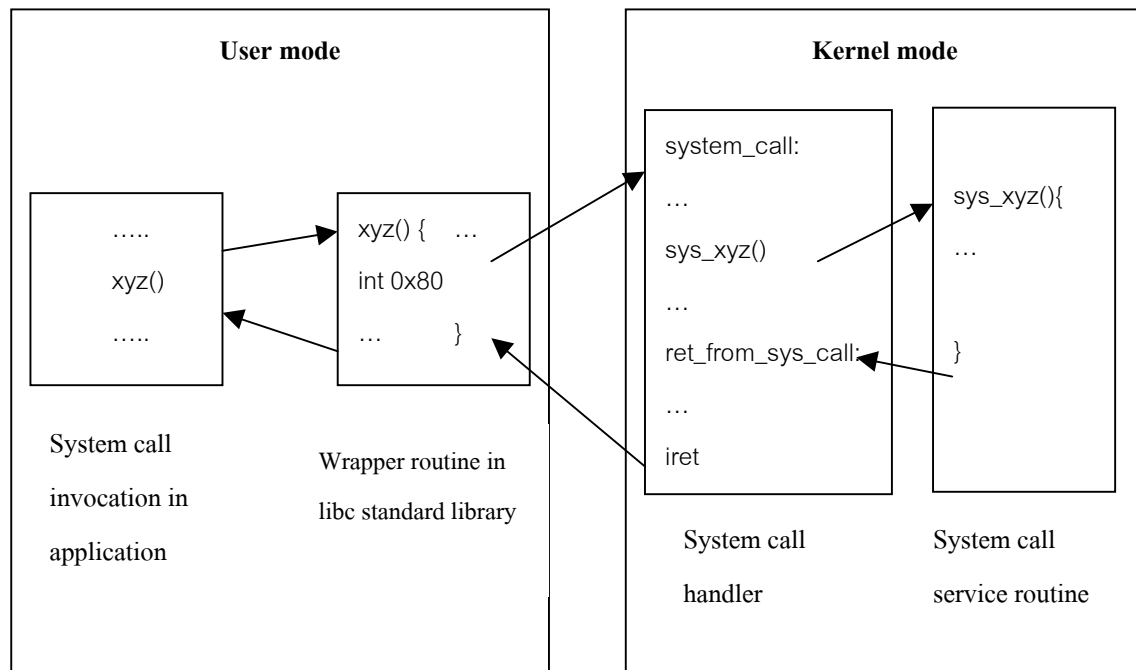
3.2 ชิสเต็มคอลล์

ระบบปฏิบัติการส่วนมากในทุกวันนี้อนุญาตให้โพรเซสต่างๆ สามารถติดต่อกับอุปกรณ์ เช่น ดิสก์ เครื่องพิมพ์ ได้จาก user mode เลยโดยมีการแบ่งเลเยอร์ระหว่างในส่วนฮาร์ดแวร์เลเยอร์ กับ แอปพลิเคชันเลเยอร์ ซึ่งในการจัดการเช่นนี้มีข้อดีอยู่หลายข้อ คือ

- ทำให้การเขียนโปรแกรมทำได้ง่ายขึ้นเพราะคนเขียนโปรแกรมไม่จำเป็นต้องเขียนโปรแกรมในระดับต่ำ
- ในแง่ของทางด้านการรักษาความปลอดภัยเพราะเคอร์เนลสามารถตรวจสอบการร้องขอได้ เพราะจำเป็นต้องร้องขอผ่านเคอร์เนล

การติดต่อกันระหว่าง user mode process กับ hardware device นั้นทำโดยการเรียกใช้ซิสเต็มคอลล์ผ่านทางเคอร์เนล สำหรับในลินุกซ์นั้นการเรียก ซิสเต็มคอลล์ นั้นทำได้โดยการอินเทอร์รัปต์หมายเลข 0x80 และส่งค่าหมายเลข ซิสเต็มคอลล์ ไว้ที่รีจิสเตอร์ eax

จากรูปเป็นการแสดงความสัมพันธ์ของการเรียก ซิสเต็มคอลล์ จากแอปพลิเคชัน เมื่อมีการร้องขอ ซิสเต็มคอลล์ผ่านจากแอปพลิเคชัน ก็จะส่งต่อไปยัง wrapper routine ที่ไลบรารี libc เพื่ออินเทอร์รับต์และ เรียกให้ system call handler ทำงานส่วนนี้ใน kernel mode ซึ่งหลังจากนั้นก็รันโค้ดซิสเต็มคอลล์ การเรียก system call routine นั้นเคอร์เนลมีตาราง system call dispatch table เก็บอยู่ในอาร์เรย์ชื่อ sys_call_table โดยสมาชิกแต่ละตัวมีตำแหน่งของ system call service routine อยู่



รูปที่ 3.1 แสดงความสัมพันธ์ของการเรียก ซิสเต็มคอลล์ จากแอปพลิเคชัน

หมายเลขซิสเต็มคอลล์เป็นตัวชี้ไปยังอาร์เรย์ชื่อ sys_call_table[] โดยหมายเลขซิสเต็มคอลล์นี้จะ ตรงกับ (map) ฟังก์ชันการให้บริการ

ตารางข้างล่างนี้เป็นตารางรายชื่อการใช้งานซิสเต็มคอลล์ที่น่าสนใจ

ซิสเต็มคอลล์	หน้าที่ของซิสเต็มคอลล์
int sys_fork(struct pt_regs regs);	ถูกเรียกเมื่อใช้คำสั่ง fork() ในยูสเซอร์สเปซ
int sys_getuid () int sys_setuid (uid_t uid)	ใช้สำหรับจัดการหมายเลข UID
int sys_get_kernel_syms (struct kernel_sym *table)	ใช้สำหรับการเข้าถึง Kernel symbol table

ตารางที่ 3.2 แสดงรายชื่อ ซิสเต็มคอลล์ ที่ใช้บ่อย

ซิสเต็มคอลล์	หน้าที่ของซิสเต็มคอลล์
int sys_sethostname (char *name,int len) int sys_gethostname(char *name,int len)	ทำการตรวจสอบค่าและเซตชื่อโฮสต์
int sys_chdir (const char *path); int sys_fchdir (unsigned int fd);	ทำการเปลี่ยนไดเรกทอรีที่ใช้งานอยู่ (คำสั่ง cd)
int sys_chmod (const char *filename, mode_t mode); int sys_chown (const char *filename, mode_t mode);	ทำการจัดการเกี่ยวกับเพอร์มิสชันของผู้ใช้
int sys_chroot (const char *filename);	เซตรูทไดเรกทอรีสำหรับโปรเซสที่กำลังทำงาน
int sys_execve (struct pt_regs regs);	รับผิดชอบในการรันไฟล์โปรแกรมต่างๆ
int sys_link (const char *oldname, const char *newname); int sym_link (const char *oldname, const char *newname); int sys_unlink (const char *name);	ทำหน้าที่ในการสร้างและลบฮาร์ดลิงค์(hardlink) และซอฟต์ลิงค์(softlink)
int sys_rename (const char *oldname, const char *newname);	ทำการเปลี่ยนชื่อไฟล์
int sys_rmdir (const char *name); int sys_mkdir (const *char filename, int mode);	ทำหน้าที่สร้างและลบไดเรกทอรี
int sys_open (const char *filename,int mode); int sys_close (unsigned int fd);	ทำหน้าที่เปิดไฟล์(รวมทั้งสร้างไฟล์ด้วย), ปิดไฟล์
int sys_read (unsigned int fd, char *buf, unsigned int count); int sys_write (unsigned int fd, char *buf, unsigned int count);	ทำหน้าที่อ่านและเขียนไฟล์
int sys_selectt (int n, fd_set *inp, fd_set *outp, fd_set *exp, struct timeval *tvp);	Multiplexing of I/O operations
sys_socketcall (int call, unsigned long args);	ทำหน้าที่เกี่ยวกับซอกเก็ตเช่น สร้างซอกเก็ต, binding, connect เป็นต้น

ตารางที่ 3.2 แสดงรายชื่อ ซิสเต็มคอลล์ ที่ใช้อยู่(ต่อ)

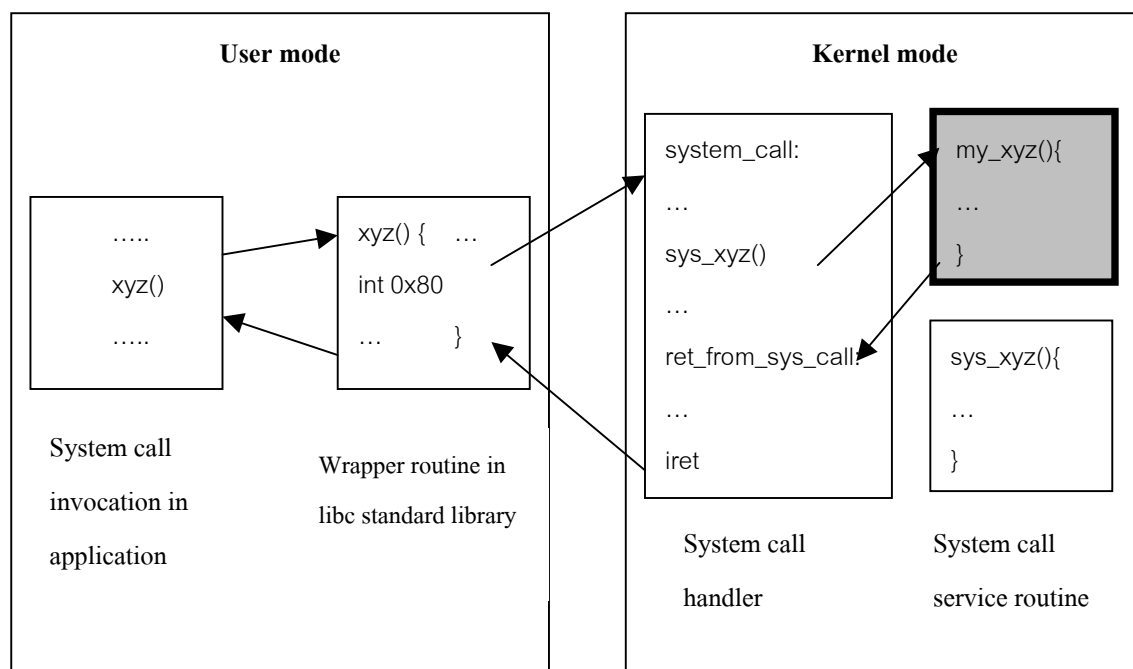
ซิสเต็มคอลล์	หน้าที่ของซิสเต็มคอลล์
<pre>unsigned long sys_create_module (char *name, unsigned long size); int sys_delete_module (char *name); int sys_query_module (const char *name, int which, void *buf, size_t bufsize, size_t *ret);</pre>	ทำหน้าที่จัดการเกี่ยวกับโมดูลเช่น โหลดโมดูล, ย้ายโมดูลออก, แสดงรายชื่อโมดูล

ตารางที่ 3.2 แสดงรายชื่อ ซิสเต็มคอลล์ ที่ใช้บ่อย

ในตารางนี้เป็นซิสเต็มคอลล์ที่ผู้บุกรุกส่วนใหญ่มักใช้ในการบุกรุก ดังนั้นจึงควรเรียนรู้การใช้ซิสเต็มคอลล์ในระบบให้ชำนาญและมีประสิทธิภาพ

3.3 การดักการทำงานของ ซิสเต็มคอลล์

จากรูปที่ 3.1 สังเกตว่าในส่วนของ kernel mode ก่อนมีการรัน system call routine นั้นต้องผ่านส่วน system call handler โดยดูตำแหน่งโปรแกรมของ system call routine ที่ตาราง sys_call_table ซึ่ง ณ จุดนี้ หากเราเปลี่ยนค่าตำแหน่งของโปรแกรมที่เก็บไว้ให้เป็นตำแหน่งของโปรแกรมที่เป็นโค้ดของเราเอง เมื่อ system call handler ทำการเรียกดูตำแหน่ง system call routine ค่าที่ได้กลายเป็นตำแหน่งของโปรแกรมของเราแทนการรัน system call routine ตามปกติก็จะมารันในโปรแกรมของเรา จุดนี้เราสามารถเขียนโปรแกรมควบคุมการทำงานได้ตามความต้องการ โดยวิธีการคือสร้างโมดูลขึ้นมาแล้วไปเปลี่ยนค่าตำแหน่งโค้ดในตาราง sys_call_table และโหลดโค้ดเข้าไปในหน่วยความจำ



รูปที่ 3.2 แสดงวิธีดักการทำงานของซิสเต็มคอลล์

จากรูปกล่องสีที่บนซ้ายมือเป็นโปรแกรมที่เราเขียนขึ้นมาให้ทำงานแทน ชิสเต็มคอลล์ เดิม โดยการเปลี่ยนค่าตำแหน่งโปรแกรมในตาราง sys_call_table ไปชี้ยังที่โปรแกรมเราแทน

อย่างไรก็ตามการดักการทำงานของ ชิสเต็มคอลล์ นั้นอาจดูเหมือนเป็นการประสงค์ร้ายต่อระบบ ซึ่งก็เป็นความจริงเพราะถ้าหากผู้บุกรุกสามารถดักการทำงานของชิสเต็มคอลล์ได้ก็จะสามารถควบคุมการทำงานของเครื่องได้อย่างมากเช่น ป้องกันการลบข้อมูลของตนโดยอาจใช้การดักการทำงานของ ชิสเต็มคอลล์ sys_unlink ขยายสิทธิ์การใช้งานของตนโดยดักการทำงานของ ชิสเต็มคอลล์ sys_setuid เป็นต้น แต่ในความจริงแล้วเราสามารถนำวิธีการดักการทำงานมาช่วยในการดูแลระบบได้เช่นกัน โดยเราต้องเลือกดักชิสเต็มคอลล์ให้ตรงกับความต้องการของสิ่งที่เราต้องการทำ

ตัวอย่างข้างล่างนี้เป็นการใช้ประโยชน์จากการดักการทำงานของชิสเต็มคอลล์โดยหากผู้ดูแลระบบต้องการป้องกันไม่ให้มีการโหลดโมดูลเข้าสู่เคอร์เนลอีก ก็สามารถทำได้โดยการดักการทำงานของชิสเต็มคอลล์ create_module ซึ่งทำหน้าที่ในการโหลดโมดูลเข้าสู่เคอร์เนล (ในคำสั่ง insmod)

```
#define __KERNEL__
#define MODULE
#include <linux/module.h>
#include <linux/kernel.h>

// ตาราง sys_call_table
extern void* sys_call_table[];
// ไว้สำหรับเก็บตำแหน่งปรกติของ ชิสเต็มคอลล์
asmlinkage int (*orig_create_module) (char *name, unsigned long size);
int my_create_module (char *name, unsigned long size){
    return 0;          // ที่ส่วนนี้โปรแกรมจะส่งค่า 0 กลับ เหมือนกับว่าไม่มี error เกิด
                        // ขึ้นเลย แต่ความจริงแล้วเรายังไม่ได้ทำการโหลดโมดูลเลย
}

int init_module(void) {
// เมื่อมีการ โหลดโมดูลนี้เข้าสู่เคอร์เนลจะทำการเรียกฟังก์ชันนี้ก่อน
// ทำการเก็บตำแหน่งของ ชิสเต็มคอลล์ ดั้งฉบับ
orig_create_module=sys_call_table[__NR_create_module];
// แทนที่ตำแหน่งโปรแกรมด้วยตำแหน่งของโปรแกรมของเรา
sys_call_table[__NR_create_module]=(void *)my_create_module;
return 0;
```

```
}  
void cleanup_module(void){  
// คืนตำแหน่งปรกติของ ชีสเต็มคอลล์ เมื่อทำการลบโมดูลออกจากหน่วยความจำ  
    sys_call_table[__NR_create_module]=orig_rmdir;  
}
```